

Session 3

Git & GitHub

Le Problème que Tout le Monde Connaît

```
projet_final.zip  
projet_final_v2.zip  
projet_final_v2_CORRIGE.zip  
projet_final_v2_CORRIGE_marche_vraiment.zip
```

La question : Comment travailler à plusieurs sur un projet sans créer le chaos ?
Comment revenir en arrière si on casse tout ?

La Solution : Git

Git n'est pas juste un système de sauvegarde. C'est une **machine à remonter le temps** pour votre code.

- **Historique complet** : Chaque changement est enregistré.
- **Collaboration propre** : Plusieurs personnes peuvent travailler en parallèle sans s'écraser.
- **Sécurité** : Vous pouvez toujours revenir à une version qui fonctionnait.

Notre mission : Maîtriser le workflow de base de Git.

Partie 1

Le Concept Clé : Les 3 Zones

Pour comprendre Git, il faut comprendre ses 3 zones.

1. Dossier de Travail (Working Directory)

- *Là où vous modifiez vos fichiers.*

2. Zone de Transit (Staging Area)

- *Là où vous préparez votre "colis" de changements. C'est le "panier d'achats" de Git.*

3. Dépôt Local (`.git` / Repository)

- *L'historique officiel et permanent de tous vos "colis" validés.*

Le flux : Modifier → Ajouter au colis → Valider le colis.

Théorie : Le Cycle de Vie d'un Fichier

1. `git add <fichier>` : Déplace un fichier du **Working Directory** vers la **Staging Area**.
2. `git commit -m "message"` : Prend tout ce qui est dans la **Staging Area** et en fait une "sauvegarde" permanente dans le **Dépôt**.

Atelier 1 : Votre Premier Voyage dans le Temps

Objectif : Créer un dépôt, faire un commit et voir l'historique.

1. Créez un dossier et initialisez Git :

```
mkdir projet-git && cd projet-git  
git init  
# Un dossier caché .git vient d'être créé !
```

2. Créez un fichier et vérifiez son statut :

```
echo "Version 1" > README.md  
git status  
# Git vous dit que le fichier est "untracked" (non suivi).
```

3. Ajoutez-le à la Staging Area et re-vérifiez :

```
git add README.md  
git status  
# Maintenant, il est prêt à être "commité" !
```

4. Créez votre premier "point de sauvegarde" :

```
git commit -m "Initial commit: Ajout du README"  
git log  
# Vous voyez votre premier commit dans l'historique !
```


Partie 2

Théorie : Git vs. GitHub

- **Git** est le **logiciel** qui tourne sur votre machine.
- **GitHub** (ou GitLab, Bitbucket) est le **site web** qui héberge vos dépôts pour les partager.

C'est la différence entre Word (le logiciel) et Google Docs (le service en ligne).

Le flux : On travaille en local, puis on **pousse** (**push**) nos changements vers le dépôt distant pour que les autres les voient.

Les Commandes de Synchronisation

- `git remote add origin <URL>`

Connecte votre dépôt local à une adresse distante (l'URL de votre dépôt GitHub). `origin` est le nom par défaut de cette connexion.

- `git push -u origin main`

Envoie (`push`) vos commits de la branche `main` locale vers le dépôt distant `origin`.

- `git pull`

Récupère les derniers changements faits par vos collègues.

Atelier 2 : Mettre son Projet en Ligne

Objectif : Pousser votre dépôt local sur GitHub.

1. **Sur GitHub.com :** Créez un nouveau dépôt vide (sans README, ni .gitignore).
2. **Copiez l'URL du dépôt** (celle qui finit en `.git`).
3. **Dans votre terminal, connectez les deux mondes :**

```
git remote add origin <URL_COPIEE_ICI>
```

4. Poussez votre premier commit :

```
# La première fois, on doit être un peu plus spécifique  
git push -u origin main
```

5. Vérifiez votre page GitHub : Votre `README.md` est apparu ! Magique.

La Super-Puissance de Git : Les Branches

Le Problème :

Comment développer une nouvelle fonctionnalité sans casser le code principal qui est en production ?

La Solution : Les Branches.

Une branche est une **copie de votre projet à un instant T**, une sorte d'univers parallèle où vous pouvez expérimenter en toute sécurité.

Théorie : Naviguer entre les Branches

- `git branch <nom-branche>` : Crée une nouvelle branche.
- `git branch` : Liste toutes vos branches.
- `git switch <nom-branche>` : Change de branche pour aller travailler dessus.

IMPORTANT : `git switch` vs `git checkout`

- `git checkout <branche>` : **L'ancienne façon**. Fait beaucoup de choses (changer de branche, restaurer des fichiers...), ce qui est confus.
- `git switch <branche>` : **La nouvelle façon (depuis 2019)**. Ne fait qu'une seule chose : changer de branche. **C'est plus clair, plus sûr. Utilisez `switch` !**

Théorie : Fusionner les Univers

Une fois que votre fonctionnalité est terminée et testée dans sa branche, il faut la ramener dans la branche principale (`main`). C'est la **fusion** (`merge`).

1. Retournez sur la branche qui doit **recevoir** les changements :

```
git switch main
```

2. Lancez la fusion :

```
git merge <nom-de-votre-branche-de-fonctionnalité>
```

Git va alors tenter d'intégrer intelligemment les changements.

Atelier 3 : Le Workflow de Branche Complet

Objectif : Créer une fonctionnalité dans une branche isolée et la fusionner.

1. Créez et basculez sur une nouvelle branche :

```
git switch -c feature/add-user-login  
# -c est un raccourci pour créer ET basculer en même temps.
```

2. Faites une modification :

```
echo "Ajout du login utilisateur" >> README.md
```

3. Faites un commit sur cette branche :

```
git add README.md  
git commit -m "FEAT: Ajout du système de login"
```

4. Retournez sur `main` et fusionnez :

```
git switch main  
git merge feature/add-user-login
```

5. Vérifiez le `README.md` et l'historique `git log` : Tout y est !

Conclusion

Le workflow moderne :

Ne jamais travailler directement sur `main`.

`Branche` → `Coder` → `Commit` → `Push` → `Merge`

C'est le rythme quotidien d'un développeur et d'un DevOps.

Questions ?